

Lab 3 — Decision Tree Learning (ID3) from Scratch

Aim

To implement the ID3 algorithm to build a decision tree using Entropy and Information Gain, and classify a new sample.

Algorithm / Procedure

1. Read the PlayTennis dataset from a CSV file.
2. Calculate the entropy of the target attribute.
3. Calculate the information gain for each available attribute.
4. Select the attribute with the maximum information gain as the decision node.
5. Divide the dataset according to the selected attribute's values.
6. Recursively repeat the process for each subset.
7. Stop when all samples belong to the same class or no attributes remain.
8. Construct and display the final decision tree.

Program

```
import pandas as pd

import numpy as np


# Create PlayTennis dataset
data = {

    "Outlook": [

        "Sunny", "Sunny", "Overcast", "Rain", "Rain",

        "Rain", "Overcast", "Sunny", "Sunny", "Rain",

        "Sunny", "Overcast", "Overcast", "Rain"

    ],

    "Temperature": [
```

```

    "Hot", "Hot", "Hot", "Mild", "Cool",
    "Cool", "Cool", "Mild", "Cool", "Mild",
    "Mild", "Mild", "Hot", "Mild"
],
"Humidity": [
    "High", "High", "High", "High", "Normal",
    "Normal", "Normal", "High", "Normal", "Normal",
    "Normal", "High", "Normal", "High"
],
"Wind": [
    "Weak", "Strong", "Weak", "Weak", "Weak",
    "Strong", "Strong", "Weak", "Weak", "Weak",
    "Strong", "Strong", "Weak", "Strong"
],
"PlayTennis": [
    "No", "No", "Yes", "Yes", "Yes",
    "No", "Yes", "No", "Yes", "Yes",
    "Yes", "Yes", "Yes", "No"
]
}

```

```
df = pd.DataFrame(data)
```

```
# Calculate Entropy
```

```
def entropy(col):

    values, counts = np.unique(col, return_counts=True)

    probabilities = counts / counts.sum()

    return -np.sum(probabilities * np.log2(probabilities))


# Calculate Information Gain
def info_gain(data, attribute, target):

    total_entropy = entropy(data[target])

    values, counts = np.unique(data[attribute], return_counts=True)

    weighted_entropy = 0

    for i in range(len(values)):

        subset = data[data[attribute] == values[i]]

        weighted_entropy += (

            counts[i] / counts.sum()

        ) * entropy(subset[target])

    return total_entropy - weighted_entropy
```

```
# ID3 Algorithm
```

```
def id3(data, attributes, target):
```

```
    # If all examples have the same class
```

```
    if len(np.unique(data[target])) == 1:
```

```
        return data[target].iloc[0]
```

```
    # If no attributes remain
```

```
    if len(attributes) == 0:
```

```
        return data[target].mode()[0]
```

```
    # Calculate information gain
```

```
    gains = {}
```

```
    for attribute in attributes:
```

```
        gains[attribute] = info_gain(
```

```
            data, attribute, target
```

```
        )
```

```
    # Select best attribute
```

```
    best_attribute = max(gains, key=gains.get)
```

```
    tree = {best_attribute: {}}
```

```
# Create branches
```

```
for value in np.unique(data[best_attribute]):
```

```
    subset = data[data[best_attribute] == value]
```

```
    remaining_attributes = [
```

```
        attribute
```

```
        for attribute in attributes
```

```
        if attribute != best_attribute
```

```
    ]
```

```
    tree[best_attribute][value] = id3(
```

```
        subset,
```

```
        remaining_attributes,
```

```
        target
```

```
    )
```

```
return tree
```

```
# Run ID3
```

```
attributes = list(df.columns[:-1])
```

```
target = df.columns[-1]
```

```
tree = id3(df, attributes, target)
```

```
print("Decision Tree:")
```

```
print(tree)
```

OUTPUT

```
*** Decision Tree:  
{'Outlook': {'Overcast': 'Yes', 'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}}, 'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}}}
```

Result

The **ID3 Decision Tree algorithm** was successfully implemented from scratch using **Entropy and Information Gain**, and the classic **PlayTennis decision tree** was obtained.